

Ответы на вопросы по RL

Неделя 1:

1. Базовые понятия RL

Обучение с подкреплением (Reinforcement Learning) — метод машинного обучения, в котором система (агент) учится принимать оптимальные решения через взаимодействие со средой. В отличие от других подходов, RL не требует заранее подготовленных данных с правильными ответами или явной структурой в них. Вместо правильных ответов в RL агент получает обратную связь (вознаграждение) за свои собственные действия.

На каждом шаге агент видит состояние s , выбирает действие a , получает награду r и переходит в новое состояние s' .

Главная цель агента — найти такую политику поведения, которая максимизирует суммарную будущую награду.

Отличия от других методов машинного обучения:

- В **обучении с учителем** модель аппроксимирует эталонные ответы, т. е. модель нуждается в размеченных данных, а действия модели не влияют на будущие входные данные;
- В **обучении без учителя** модель ищет скрытые взаимосвязи, зависимости и структуры в данных без какой-либо обратной связи или меток;
- В **обучении с подкреплением** агент сам влияет на свои последующие наблюдения через собственные действия. Обучение происходит через исследование среды и использование накопленного опыта.

Основные понятия:

- **state (s)** — состояние среда, информация о текущей ситуации, доступная агенту;
- **action (a)** — действие, которое агент выбирает совершить в состоянии **state**;
- **reward** — численный сигнал качества совершённого действия **action**;
- **policy** — правило выбора действия: детерминированное ($a = \pi(s)$) или стохастическое ($\pi(a | s)$);
- **trajectory / episode / session** — последовательность состояний, действий и наград до завершения эпизода;
- **return** — суммарная награда за эпизод;
- **discount factor gamma** — коэффициент важности будущих наград;

- $V(s)$ - ожидаемая будущая награда из состояния s ;
- $Q(s, a)$ - ожидаемая будущая награда после действия a в состоянии s ;
- **exploration** - пробование новых действий;
- **exploitation** - использование уже найденной хорошей стратегии;

2. Метод кросс-энтропии (табличный + нейронная сеть)

Метод кросс-энтропии — итеративный алгоритм оптимизации политики, который относится к категории методов прямого поиска. Общая идея метода заключается в циклическом повторении трёх шагов — генерация сессий, отбор лучших сессий и подстройка политики под эти сессии.

Вообще говоря, метод кросс-энтропии позволяет решить RL-задачу через подбор лучших политик.

Общий алгоритм выглядит следующим образом:

- Инициализация стохастической политики (например, случайными весами или равномерным распределением);
- Циклически повторять:
 - Генерация множества эпизодов;
 - Выбор элитных эпизодов с наградой выше заданного значения **percentile**;
 - Обновление политики таким образом, чтобы действия из элитных политик стали более вероятными.

В случае, если пространство состояний конечно и невелико, политику можно представить как матрицу вероятностей, где строками будут являться состояния, а столбцами действия. Такой подход является табличным методом кросс-энтропии.

Алгоритм табличного метода кросс-энтропии выглядит следующим образом:

- Равномерная инициализация политики;
- Генерация **n_sessions** эпизодов;
- Подсчёт награды **reward** каждого эпизода;
- Поиск порогового значения **reward_threshold = percentile(rewards)**;
- Получения состояний и действий из элитных эпизодов;
- Обновление вероятностей действий по частотам в элитных данных;
- Полное повторение цикла до нахождения «хорошего» среднего **reward**.

Плюсы метода: простота, понятная отладка, хорошая работа на маленьких дискретных задачах.

Минусы: требуется много сэмплов, плохо масштабируется на большие пространства состояний, политика может слишком рано стать почти детерминированной.

В задачах с огромным или бесконечным пространством состояний табличный подход неприменим. В таких случаях политика аппроксимируется параметрической моделью (например, нейронной сетью). Такой подход называется нейросетевым методом кросс-энтропии.

Идея остается той же: повысить вероятность действий, которые встречались в успешных эпизодах. Отличие в том, что нейросеть умеет обобщать между похожими состояниями, поэтому нейросетевой метод подходит для непрерывных пространств состояний.

Важные замечания:

- действия нужно сэмплировать из вероятностей, а не всегда брать **argmax**;
- **percentile** управляет жесткостью отбора элиты;
- **n_sessions** влияет на стабильность оценки лучших траекторий;
- для непрерывных состояний полезна нормализация;
- **replay** нескольких последних итераций и **epsilon-исследование** помогают не потерять **exploration** слишком рано.

Неделя 2:

1. $V(s)$, $Q(s, a)$ – что это такое?

В обучении с подкреплением агент, находясь в состоянии s , выполняет действие a , затем получает награду и переходит в следующее состояние s' . Иногда объектом интереса является не только ближайшая награда, но и вся будущая суммарная награда. В RL главной целью является максимизация ожидаемого суммарного вознаграждения, называемого возвратом (**return**).

Возврат — это сумма вознаграждений, полученных агентом, начиная с времени t до конца эпизода. В задачах с бесконечным горизонтом (где нет чёткого конца эпизода) простая сумма может стремиться к бесконечности даже при субоптимальном поведении. Чтобы избежать этого и математически формализовать предпочтение немедленных будущим, вводится коэффициент дисконтирования (**gamma**).

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

где $0 \leq \gamma < 1$.

$V^\pi(s)$ – функция ценности состояния. Она показывает какую ожидаемую суммарную награду получит агент, если он стартует из состояния s и дальше действует по политике π :

$$V^\pi(s) = E_\pi[G_t : S_t = s].$$

Т.е. функция ценности состояния отвечает на вопрос насколько выгодно находится в состоянии s , если дальше следовать политике π .

$Q^\pi(s, a)$ – функция ценности действия. Она показывает какую ожидаемую суммарную награду получит агент, если он, находясь в состоянии s , сначала выполнит действие a , а затем продолжит действовать по политике π :

$$Q^\pi(s, a) = E_\pi[G_t : A_t = a].$$

Т.е. функция ценности действия отвечает на вопрос насколько выгодно выбрать действие a , находясь в состоянии s .

Связь между этими функциями:

$$V^\pi(s) = \sum_a \pi(a : s) Q^\pi(s, a).$$

Если политика детерминированная, она выбирает одно действие $\pi(s)$, и тогда:

$$V^\pi(s) = Q^\pi(s, \pi(s)).$$

Для оптимальных функций ценности:

$$V^*(s) = \max_a Q^*(s, a).$$

Оптимальная политика выбирает действие с максимальным Q^* :

$$\pi^*(s) \in \operatorname{argmax}_a Q^*(s, a).$$

2. Метод Policy iteration

Главная цель обучения с подкреплением — найти оптимальную политику π , которая максимизирует ожидаемый возврат во всех состояниях. Метод Policy Iteration реализует концепцию обобщённой итерации политики, которая состоит из чередования двух процессов: оценки и улучшения.

Пусть у нас есть текущая политика π . На первом шаге вычисляется функция ценности $V^\pi(s)$ для всех состояний. Для детерминированной политики эта функция удовлетворяет уравнению Беллмана:

$$V^\pi(s) = \sum_{s'} P(s' : s, \pi(s)) [r(s, \pi(s), s') + \gamma V^\pi(s')].$$

После этого на втором шаге улучшаем политику, выбирая более выгодные действия — в каждом состоянии выбираем действие, которое даёт максимальное значение с учётом найденной функции V^π :

$$\pi_{new}(s) \in \arg \max_a \sum_{s'} P(s' : s, a) [r(s, a, s') + \gamma V^\pi(s')].$$

Если после улучшения политика не изменилась, значит она уже оптимальна.

Полный алгоритм Policy Iteration:

- Инициализировать произвольную политику π_0 ;
- Для текущей политики π_k вычислить V^{π_k} ;
- Построить новую жадную политику π_{k+1} по V^{π_k} ;
- Повторять пока политика не перестанет меняться.

Неделя 3:

1. Q-learning, SARSA

В реальных задачах мы редко обладаем полной информацией о динамике среды. Это заставляет нас переходить от динамического программирования к методам Model-free, где агент обучается на основе реального или симулированного опыта — траекторий — последовательностей состояний, действий и наград.

Оба алгоритма — Q-learning и SARSA базируются на идее TD-обучения. Вместо того, чтобы ждать конца эпизода, агент обновляет свои оценки на каждом шаге, используя текущую награду и оценку ценности следующего состояния.

Q-learning и SARSA это Model-free TD-методы. Они не знают вероятности переходов среды и учатся напрямую из опыта. Оба метода оценивают $Q(s, a)$, т. е. ожидаемую будущую суммарную награду после выбора действия **a** в состоянии **s**.

Q-learning это метод обучения вне политики (off-policy). Это значит, что агент учится оптимальному поведению независимо от того, какие действия он совершает в процессе исследования. Для обновления $Q(s, a)$ алгоритм использует жадную оценку следующего состояния, предполагая, что в будущем он выберет наилучшее возможное действие:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)].$$

Здесь используется максимум по действиям в следующем состоянии. Поэтому Q-learning как будто бы предполагает, что на следующих шагах агент будет действовать оптимально.

SARSA это on-policy метод, т. е. агент оценивает именно ту политику, которой он следует в данный момент (включая все случайные исследовательские действия). Обновление происходит по формуле:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)].$$

SARSA расшифровывается как State-Action-Reward-State-Action, т. е. происходит от кортежа:

$$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}).$$

Главное отличие двух этих методов в том, что SARSA использует реально выбранное следующее действие a_{t+1} , а Q-learning использует лучшее возможное действие через максимум.

2. Exploration vs Exploitation

Одной из главных проблем обучения с подкреплением является баланс между использованием уже накопленных знаний (exploitation) и поиском новых, потенциально лучших решений (exploration).

Exploitation означает использовать уже найденную хорошую политику — выбирать действие с максимальной текущей оценкой $Q(s, a)$. Так агент в моменте получает высокую награду, но может застрять в локально хорошей политике и не найти лучшую.

Exploration означает иногда пробовать другие действия, даже если сейчас они кажутся хуже. Это нужно потому, что оценки $Q(s, a)$ в начале обучения неточные.

Например, есть epsilon-greedy политика:

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A|}, & \text{если } a \in \arg \max_b Q(s, b), \\ \frac{\epsilon}{|A|}, & \text{иначе.} \end{cases}$$

Если ϵ большое, значит агент много исследует. Если маленькое, значит агент в основном использует текущую лучшую стратегию. Обычно в начале обучения exploration делают сильнее, а затем постепенно уменьшают ϵ .

Это называется epsilon decay. Идея в том, чтобы сначала собрать разнообразный опыт, потом стабилизировать поведение и использовать найденную политику.

Неделя 4:

1. DQN

Классический табличный Q-learning метод предполагает хранение значений функции $Q(s, a)$ для каждой пары состояние-действие. Однако в реальных задачах пространство состояний может быть огромным или непрерывным. Для решения этой проблемы используется DQN.

DQN (Deep Q-learning) – вариант Q-learning, где таблица значений $Q(s, a)$ заменяется нейронной сетью. В табличном варианте мы храним отдельное значение для каждой пары состояние-действие, но если мы работаем, например, с изображениями, то количество возможных состояний бесчисленно (поскольку бесчисленно множество всевозможных изображений).

Поэтому DQN обучает нейронную сеть:

$$Q_{\theta}(s, a),$$

которая по входному состоянию s предсказывает значения функции ценности действия для всех действий.

Цель обучения похожа на обычный Q-learning. Для перехода (s, a, r, s') строится целевая функция:

$$y = r + \gamma \max_{a'} Q_{\theta'}(s', a'),$$

где θ' - параметры целевой нейронной сети.

Функция потерь:

$$L(\theta) = E[(Q_{\theta}(s, a) - y)^2].$$

2. Experience replay

В стандартном обучении нейронных сетей предполагается, что обучающие примеры являются независимыми и одинаково распределёнными. В обучении с подкреплением это условие нарушается: последовательные кадры в игре (траектория агента) сильно коррелируют между собой. Кроме того, если агент долго находится в одной локации, сеть быстро забывает как вести себя в других частях среды, что приводит к деградации кривой обучения.

Для того, чтобы разорвать временные связи, предлагается сохранять прошлые взаимодействия агента со средой в виде кортежей (s, a, r, s') в специальную память — Replay Buffer — буфер старых переходов агента.

Во время взаимодействия со средой агент складывает переходы в Replay Buffer. При обучении он берет случайный mini-batch из этого буфера, а не только последний переход. Это нужно по нескольким причинам:

1. Соседние переходы в обучении с подкреплением сильно коррелированы. Если учить сеть подряд на почти одинаковых состояниях, обучение становится нестабильным. Replay Buffer перемешивает опыт и делает mini-batch более похожим на обычный supervised learning dataset;
2. Один и тот же опыт используется несколько раз – агент получает больше обучающих шагов из уже собранных данных;
3. Replay Buffer сглаживает изменения распределения данных. Политика агента постоянно меняется, а буфер содержит смесь старого и нового опыта, поэтому обучение становится менее резким.

3. Target network

В базовом DQN одна и та же сеть используется и для предсказания текущего значения $Q(s, a)$, и для вычисления значения целевой функции. Это создаёт проблему — когда мы обновляем веса, чтобы приблизить $Q(s, a)$ к цели, сама цель тоже сдвигается, т. к. веса обновились. Это часто приводит к расходимости алгоритма.

Решение проблемы — ввод второй нейронной сети — Target Network с параметрами θ^- . Она используется исключительно для вычисления целевого значения:

$$L(\Theta) = E_{s \sim S, a \sim A} [(Q(s, a, \Theta) - (r + \gamma \max_{a'} Q(s', a', \Theta)))^2].$$

Параметры целевой сети Θ остаются фиксированными (замороженными) на протяжении многих шагов обучения.

4. Double DQN

Использование оператора $\max$ в целевом значении Беллмана приводит к систематическому завышению оценок Q-значений. Это происходит из-за того, что даже если истинные ценности действий равны, случайный шум в аппроксимации может сделать одно действие чуть лучше других, и именно оно будет выбрано оператором $\max$.

Для решения данной проблемы необходимо разделить процесс выбора лучшего действия и процесс оценки его стоимости. В Double DQN это реализуется через использование двух сетей (основной и целевой):

1. Основная сеть Θ выбирает действие, которое она считает лучшим;
2. Целевая сеть Θ' оценивает стоимость этого выбранного действия.

Полная формула:

$$y = r + \gamma Q_{\theta'}(s', \arg \max_{a'} Q_{\theta}(s', a'))$$

Такой подход значительно снижает смещение оценок и позволяет агенту сходиться к более стабильным и реалистичным стратегиям.

Неделя 6:

1. Policy gradient

Policy gradient – это семейство методов, которые оптимизируют политику напрямую. Ранее рассматривались методы, основанные на ценности, такие как Q-learning, где агент сначала учит функцию ценности, а затем выводит политику через $a = \arg \max Q(s, a)$. Однако вычисление точных Q-значений зачастую оказывается более сложной задачей, чем непосредственный выбор оптимального действия.

В Policy gradient сразу задаётся параметризованная политика:

$$\pi_{\theta}(a | s),$$

где θ - параметры модели, например, веса нейронной сети. Основная цель — максимизировать ожидаемую суммарную награду:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [G(\tau)],$$

где τ - траектория, собранная политикой π_{θ} , а $G(\tau)$ - возврат.

Если действие привело к большой будущей награде, то мы увеличиваем его вероятность. Если возврат плохой, то вероятность такого действия уменьшается.

Главная формула Policy gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t].$$

2. REINFORCE

Алгоритм REINFORCE является классической реализацией метода Policy gradient, основанной на методе Монте-Карло. Он работает следующим образом:

1. Сгенерировать один или несколько эпизодов текущей политикой π_{θ} ;
2. Для каждого шага посчитать возврат $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$;
3. Обновить параметры политики в направлении:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t.$$

В данном методе есть ряд ограничений.

Во-первых, REINFORCE является on-policy методом, а значит он требует, чтобы данные были получены строго по текущей политике. Мы не можем использовать старый опыт, т. к. градиент логарифма вероятности привязан к конкретному распределению.

Во-вторых, особенностью является высокая дисперсия — поскольку оценка градиента зависит от полной суммы наград G_t , которая может сильно варьироваться от сессии к сессии, обучение часто бывает нестабильным.

Для стабилизации обучения часто используют Baseline:

$$\nabla_{\theta} J(\theta) \approx \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b(s_t))].$$

Baseline не меняет математическое ожидание градиента, но уменьшает его дисперсию.

3. Actor-critic

Метод Actor-Critic – это гибридный подход, объединяющий преимущества методов на основе ценности и градиента стратегии. Он объединяет policy-based и value-based подходы. В нём есть две части:

1. Actor – политика $\pi_{\theta}(a | s)$, которая выбирает действия;
2. Critic – функция ценности, например, $V_{\phi}(s)$ или $Q_{\phi}(s, a)$, которая оценивает качество состояний или действий.

Actor обновляется методом Policy gradient, но вместо полного метода Монте-Карло возврат использует оценку преимущества $A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$.

На практике часто используют TD-error как приближение $\delta_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$. Тогда функция потерь для Actor:

$$L_{actor} = -\log \pi_{\theta}(a_t | s_t) \delta_t.$$

Critic обучается предсказывать функцию значений, например, минимизируя TD потери:

$$L_{critic} = (r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t))^2.$$

Интуитивное восприятие:

- Actor отвечает за то, что делать;
- Critic говорит, насколько действие оказалось лучше или хуже ожидания.

Неделя 7:

1. Применение RNN для генерации последовательностей

Рекуррентные нейронные сети (RNN) являются фундаментальным инструментом для работы с последовательными данными, такими как временные ряды, аудиосигналы и тексты. В отличие от стандартных нейронных сетей прямого распространения, RNN обладают памятью, что позволяет им учитывать контекст всей предыстории при принятии решений на текущем шаге.

RNN применяют для генерации последовательностей тогда, когда следующий элемент зависит от уже существующего сгенерированного префикса. Основная задача генерации последовательностей при работе с текстом — это построение языковой модели.

Модель задаёт распределение:

$$p_{\theta}(y_1, \dots, y_T | x) = \prod_{t=1}^T p_{\theta}(y_t | y_{<t}, x).$$

В seq2seq-задачах обычно имеется encoder и decoder. Encoder читает входную последовательность x и превращает её в скрытое представление. Decoder генерирует выходную последовательность по одному токёну:

$$\begin{aligned} h_t &= RNN(h_{t-1}, y_{t-1}), \\ p_{\theta}(y_t | y_{<t}, x) &= \text{softmax}(Wh_t). \end{aligned}$$

При обучении часто используют teacher forcing, на вход decoder подается настоящий предыдущий токён. При генерации настоящих токёнов уже нет, поэтому decoder использует собственный предыдущий output. Токёны можно выбирать greedy-способом через argmax , либо сэмплировать из распределения.

Обучение генеративных RNN методом обратного распределения ошибки во времени сопряжено с проблемами взрывающихся и затухающих градиентов. Для решения таких проблем используются архитектуры с пропусками связей, которые позволяют градиентам течь через время почти без затухания, и Gradient Clipping – принудительное ограничение нормы градиента для предотвращения расходимости.

2. Применение RL к дообучению моделей генерации последовательностей

Стандартное обучение моделей генерации методом максимизации правдоподобия имеет ряд существенных ограничений, которые решаются с помощью методов обучения с подкреплением.

При обучении с учителем модель на каждом шаге получает идеальный предыдущий токён из обучающей выборки. Однако во время генерации модель вынуждена опираться на свои собственные, возможно ошибочные, предсказания. Малейшая ошибка на раннем этапе приводит к выходу из распределения данных, и сеть начинает генерировать мусорные значения. Обучение с подкреплением решает эту проблему, т. к. в этом режиме модель обучается непосредственно на своих собственных выводах.

Существует несколько различных подходов:

1. Самокритичное обучение — подход в рамках градиента политики, который использует режим инференса как базовую линию для снижения дисперсии;
2. Метод обучения на обратной связи от людей. Современные большие языковые модели проходят через трёхэтапный процесс дообучения:
 - Дообучение на качественных примерах «инструкция — ответ». Одной только этой стадии недостаточно, т. к. она провоцирует галлюцинации — модель учится имитировать ответ даже если не знает фактов;
 - Reward Model (RM) – обучение нейронной сети-критика, которая предсказывает оценку ответа человеком на основе ранжирования нескольких вариантов;
 - RL Fine-Tuning (PPO) – модель генерации (Actor) оптимизируется против RM с использованием алгоритма PPO, чтобы давать более полезные и безопасные ответы.